

Verification Continuum™

Verdi® and Siloti® Quick Reference Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	5
Synopsys Statement on Inclusivity and Diversity	5

1. Verdi and Siloti Quick Reference Guide	7
Automatic Tracing of Value with Verdi's Temporal Flow View	8
Automatic Tracing of X's with Verdi's Temporal Flow View	8
Active Tracing of RTL	9
Function Debugging	9
Macro Debugging	9
SystemVerilog TestBench (SVTB) – FSDB Logging	9
Compile and Dump SystemVerilog Assertions	10
Compute Newly Added SVA in Verdi Without Re-running Simulation	11
Analyze the Reason for Assertion Failures	11
Shift Time for Signals	12
Count the Transitions of a Clock or Register	12
Modify Existing Logical Operation Signals	12
Trace Memory Contents Without Re-running Simulators	13
Compute Memory Contents and Write to an FSDB File	13
Virtually Combine Multiple FSDB Files as a Single FSDB	14
Aliasing and Alias Files	14
Toggle Coverage Analysis	15
Compare Two FSDB Mismatches	15
Compress Time Region	16
Collapse Source Code	17
nAnalyzer - Extract Clock Tree	17
nAnalyzer - Switching Reports	17
nAnalyzer - Timing Analysis	19
nECO - Graphical Engineering Change Order Enhancement Tool - Modify Gate Level	
Netlist - Non-Freeze Silicon ECO	19

Contents

Accelerate Simulation and Reduce Dumping Size with Siloti21

Resources 22

Preface

The Verdi and Siloti Quick Reference Guide provides a quick reference for using Verdi and Siloti systems with typical debug scenarios.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also

Contents

includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Verdi and Siloti Quick Reference Guide

The following sections provide a quick reference for using the Verdi and Siloti systems with typical debug scenarios:

- [Automatic Tracing of Value with Verdi's Temporal Flow View](#)
- [Automatic Tracing of X's with Verdi's Temporal Flow View](#)
- [Active Tracing of RTL](#)
- [Active Tracing of RTL](#)
- [Function Debugging](#)
- [Macro Debugging](#)
- [SystemVerilog TestBench \(SVTB\) – FSDB Logging](#)
- [Compile and Dump SystemVerilog Assertions](#)
- [Compile and Dump SystemVerilog Assertions](#)
- [Compute Newly Added SVA in Verdi Without Re-running Simulation](#)
- [Analyze the Reason for Assertion Failures](#)
- [Shift Time for Signals](#)
- [Count the Transitions of a Clock or Register](#)
- [Modify Existing Logical Operation Signals](#)
- [Trace Memory Contents Without Re-running Simulators](#)
- [Compute Memory Contents and Write to an FSDB File](#)
- [Virtually Combine Multiple FSDB Files as a Single FSDB](#)
- [Aliasing and Alias Files](#)
- [Toggle Coverage Analysis](#)
- [Compare Two FSDB Mismatches](#)
- [Compress Time Region](#)

- [Collapse Source Code](#)
- [nAnalyzer - Extract Clock Tree](#)
- [nAnalyzer - Extract Clock Tree](#)
- [nAnalyzer - Switching Reports](#)
- [nAnalyzer - Timing Analysis](#)
- [nECO - Graphical Engineering Change Order Enhancement Tool - Modify Gate Level Netlist - Non-Freeze Silicon ECO](#)
- [Accelerate Simulation and Reduce Dumping Size with Siloti](#)
- [Resources](#)

Automatic Tracing of Value with Verdi's Temporal Flow View

1. In the *nWave* frame, find a signal that you want to debug.
2. Select the signal.
3. Click the **Auto Trace** toolbar icon .

or

Right-click to invoke **Temporal Flow View -> Create Temporal Flow View**.

4. In the *Temporal Flow View* frame, select a port signal, right-click to invoke **Trace This Value**.

If the value of multiple signals is traced, you may find that some paths are merged into the same path for a better high level view.

Automatic Tracing of X's with Verdi's Temporal Flow View

1. In the *nWave* frame, select a signal that has transitioned to an X.
 2. Click **Auto Trace** toolbar icon .
- or
- Right-click to invoke **Temporal Flow View -> Create Temporal Flow View**.
3. In the *Temporal Flow View* frame, select a port signal, right-click to invoke **Trace Active X**.
 4. In the *Trace X Settings* form, set the tracing constraint and click the **Trace** button

Active Tracing of RTL

1. In the *nWave* frame, select a specific signal that you want to debug.
2. Double-click a transition point that you want to debug.
This takes you to the line in source code that is causing the transition.
3. In the main window, invoke **Source -> Active Annotation**.
4. Click on the signal, right-click to invoke **Active Trace**.

Function Debugging

1. In the source code frame, find a signal that is driven by a function, for example, “add” and so on.
2. Invoke **Source -> Function Annotation**.
3. Double-click the function, for example, “add”.
4. Click on function, for example. “add” to jump back.

Macro Debugging

1. In the source code frame, find a macro, for example. MACRO1(CLOCK2).
2. Put your mouse cursor on MACRO1. The tip window shows the definition of MACRO1.
3. Invoke **Source -> Expand Macro**.

SystemVerilog TestBench (SVTB) – FSDB Logging

1. The `$fsdbLog` dumping command can be used to log messages into the FSDB file. The use of `$fsdbLog` is not restricted to SVTB and can be used anywhere in your environment where you have previously used crude text logging mechanisms to log interesting information. The flexibility of `$fsdbLog` allows you to capture not only messages but also severities, variable states, and so on.
2. The messages are recorded in the **msg_root** scope in the FSDB file. Invoke the **Signal -> Get Signals** command in *nWave* to get them.
3. Select a message stream in the waveform frame, right-click the message and choose **Properties** command to view property details.

4. Use the **Waveform -> Message -> Expand/Shrink Overlapping** command to expand the overlapped messages.
5. Use the **Waveform -> Message -> Filter/Colorize** command to filter and colorize messages.

Compile and Dump SystemVerilog Assertions

1. Adding dumping task to design:

- `$fsdbDumpSVA;` command is required to dump SVA data. Only failure asserts are dumped, the Verdi system calculates properties automatically when analyzing.
 - Adding the `+fsdb+sva_success` simulation option, if success SVA needs to be dumped.
- For VCS, include the `-debug_access+<option>` compile-time option on the VCS command line.
 - Include the `-sverilog` option to enable SystemVerilog features.

2. Compiling using UFE:

- Include the `-kdb` compile-time option to generate Verdi KDB with Unified Compile front end and to dump the design into the libraries specified in the `synopsys_sim.setup` file. For example:

```
// -kdb in VCS two-step flow
% vcs -kdb <compile_options> <source files> -debug_access+all -lca
// -kdb in VCS three-step flow
%> vlogan -kdb <vlogan options> <source files>
%> vhdlan -kdb <vhdlan options> <source files>
%> vcs -kdb <top_name> -debug_access+all -lca
```

3. Importing the KDB:

```
// Invoke in Interactive Mode
% simv -verdi
// Invoke in Post-processing Mode
% simv
% verdi -ssf <the dumped FSDB> //The KDB will be imported
  automatically
```

or

```
% Verdi -dbdir ./simv.daidir //To load the generated KDB only
```

Compute Newly Added SVA in Verdi Without Re-running Simulation

1. Load design and signal level FSDB into the Verdi system.
2. In the main window, invoke **Tools -> Property Tools -> Evaluator**.
 - a. In the *Evaluate Properties* form, expand the design hierarchy in the left pane, and click the node that contains assertions.
 - b. Select assertions in the middle pane, and click the **Add Selected Properties** icon  to add assertions to the right pane.
 - c. Click the **Evaluate** button, and click **Yes** on the *Question* box.

The assertions are evaluated, and a virtual FSDB file is generated to combine the original FSDB file and the new created one. The virtual FSDB file will be loaded into *nWave* automatically.

Analyze the Reason for Assertion Failures

1. In the main window, invoke **Tools -> Property Tools -> Statistics** to open the *Statistics* frame.
2. Double-click **Assert** in the **FSDB Statistics** section to add all assertions to the **Property Details** field.
3. Then double-click an assertion failure in the **Property Details** field. The *Analyzer* frame appears at the bottom.
4. Scroll down the *Analyzer* frame to view property and sequences for the specific SV Assertion that you want to debug.

Notice the time information above and the data information below the variable signals in the sequences.

5. Click the **Expand** button .

This focuses in on the logical failures in the sequence.

6. In the *nWave* frame, select the SV Assertion that you want to debug and invoke **Waveform -> Property -> Expand Overlapping/Shrink Overlapping**.
7. View the expanded assert at top for the specific SV assertion that you want to debug.

Shift Time for Signals

1. To shift the complete FSDB file:
 - In *nWave*, invoke **Waveform -> Waveform Time -> Shift File Time**.
2. To shift just one signal's time:
 - In *nWave*, invoke **Waveform -> Waveform Time -> Shift Individual Signal Time**.
3. Enter the time (+ or -) to shift the FSDB file or signal by, for example: *500*.

Count the Transitions of a Clock or Register

1. To turn on the grid count in *nWave*:
 - a. In *nWave*, invoke **View -> Grid Options**.
 - b. In the *Grid Options* form, enable the **Grid on**, **Rising Edge**, and **Grid Count with Start Number** options and click the **Apply** button at the bottom.
 - c. Left-click in the *nWave* frame to move the start count on the signal that you want to count.
2. To jump to the 100th rising edge, for example:
 - Go back to the *Grid Options* form and click on the **Lock Grid Count** and **Jump Cursor to Grid Number**, type **100** in the text field and click the **Apply** button at the bottom.
3. From the same *Grid Options* form, disable the **Grid on** and **Rising Edge** options, then click the **Apply** button at the bottom to remove all grids.

Modify Existing Logical Operation Signals

1. In *nWave*, left-click a signal that you'd like to create a copy/modify.
2. In *nWave*, invoke **Signal -> Logical Operation**, and then select an existing logical operation signal in the *Logical Operation* form.
3. Modify the content in the **Expression** field, for example, if you have selected an 8 bit register, perform a logical right two bit shift operation like this: **"top/reg1[7:0]" >> 2**
4. Click the **Create/Modify** button.

Trace Memory Contents Without Re-running Simulators

1. From *TFV*, invoke **Tools -> Show Memory Contents**.

-or-

From the source code frame, right-click the memory variable to invoke **Debug Memory -> Show Memory Contents**.

2. In the *Get Memory Variable* form, click the **Calculate by Verdi** tab and specify the display range and time then click **OK**.
3. In the *nMemory* frame, invoke **Time -> Sync Cursor Time** to automatically locate the last write.

Compute Memory Contents and Write to an FSDB File

1. Find location for memory array. In the *Source Code* pane, right-click to invoke **Debug Memory -> Dump Memory Waveform to FSDB**.
 - a. In *Dump Memory Waveform to FSDB* form, be sure to set start time and end time and enter name for **Dump FSDB File** similar as "**my_memory_dump.fdsb**",
 - b. Then click **Start Dumping** button.
2. In *nWave*, invoke **File -> Edit Virtual File**.
 - a. In *Virtual File Editor* form, select the original FSDB file and the new "**my_memory_dump.fdsb**" file and click the **Add** button.
 - b. Then click the **OK** button.
3. In *nWave*, reload the created virtual file (default name will be **VirtualFile.vf**) by the **File -> Open** command.
4. Find the location for the memory array. In the source code frame, right-click to invoke **Debug Memory -> Show Memory Contents**.
 - Then in *Get Memory Variable* form, select **Dumped by Simulator** tab, find your memory array name in right side of form. Select it and then click **OK** button.
5. Finally, in *nMemory* frame, click the search arrow buttons to find changes of last write in memory array. These changes are shown in red.

Virtually Combine Multiple FSDB Files as a Single FSDB

1. Open a new *nWave* frame.
 - In the main window, click **New Waveform** toolbar icon
 - or-
 - Invoke **Tools -> New Waveform** from pull-down menu.
2. In *nWave*, invoke **File -> Edit Virtual File**.
 - In the *Virtual File Editor* form, select an FSDB file and click the **Add** button. Add more files in the same way, and then click the **OK** button.
3. In *nWave*, reload the created virtual file (default name is **VirtualFile.vf**) by **File -> Open** command.
4. If you already have many other FSDB files opened in the current *nWave* frame, you have to set the new virtual file as active:
 - a. In *nWave*, invoke **File -> Set Active**.
 - b. In the *Active File* form, select the virtual file that you just created and click the **OK** button.

Aliasing and Alias Files

1. Use the `-autoalias` option on the Verdi command line for automatic mnemonic recognition for 'defines and parameters'.

```
% verdi -f run.f -autoalias
```
2. Use `aliasextract` option to extract an alias file from a compiled library. The default is `extracted.alias`.

```
% vericom -f run.f -lib work
% aliasextract -lib work
```
3. Use the `-aliasFile` option on the Verdi command line to load an alias file.

```
% verdi -top system -aliasFile extracted.alias
```

4. To add an alias file from *nWave*:
 - a. In *nWave*, click a bus signal in the hierarchy pane.
 - b. Invoke **Waveform -> Signal Value Radix -> Add Alias from File**.
 - c. The **.alias** file contains lines that look as following:

Then select the **.alias** file, and click **OK** button.

```
ADDA      6'h0d
ADDB      6'h0e
```

Toggle Coverage Analysis

1. Load as many FSDB files as you like:
 - a. In *nWave*, invoke **File -> Open**.
 - b. Select an FSDB file and click the **Add** button (select more if needed).
 - c. Then click the **OK** button.
2. In *nWave*, invoke **Tools -> Toggle Coverage Report**.
 - a. In the *Toggle Coverage* form, your FSDB files are shown in the **Target File(s)** section.
 - b. Select **Full**, **Partial** or **Any Change** in **Toggle Criterion** section.
 - c. Then click **Apply** button.
 - d. Click the **Report** button to see the toggle report.
3. In the *Toggle Coverage Report* form, select **Toggled** or **Not Toggled** in the **List by** section.
4. To save report, click the **Save** button and save the result as "something.rpt".

Compare Two FSDB Mismatches

1. Open two *nWave* frames with different FSDB files. Ideally pass vs. fail FSDB files on same test or RTL vs. Gate simulation on locked in signal names.
 - In the first *nWave* frame, invoke **File -> Open**, select an FSDB file and click the **Add** button, then click **OK** button.
 - In the second *nWave* frame, invoke **File -> Open**, select a different FSDB file and click the **Add** button, then click **OK** button.

2. Put the same signals to the two *nWave* frames. You can drag and drop signals from one *nWave* frame to another.
 3. Synchronize the *nWave* frame cursors.
 - In the first *nWave*, invoke **Window -> Sync Waveform View**.
 - In the second *nWave*, invoke **Window -> Sync Waveform View**.
 4. Compare any parts of the signals, for example. compare all signals in each group for both *nWave* frames.
 - a. In *nWave*, invoke **Tools -> Waveform Compare -> Compare Two Groups**.
 - b. In *Compare Two Groups* form, click **Groups in Different Windows**.
 - c. Select the first group name from the first **Group Name** selection field. And select the second group name from the second **Group Name** selection field.
 - d. The *Comparison Result* form shows your comparison results for the signals in each group.
 - e. To save report, click **Save** button to save the report as a .txt file.
 5. Search by mismatches:
 - In the first *nWave* frame, make sure **Search By** option is set to **Mismatches (/)**, then click on the blue horizontal arrows next to the **Search By** option.
-

Compress Time Region

1. In *nWave*, invoke **View -> Compress Time Range**.
 - In the *Compress Time Range* form, enter the value in the **From Time** and **To Time** fields.
 - OR-
 - Go to *nWave* frame, left-click a starting time and click middle mouse button on an ending time, then go back to *Compress Time Range* form and click **Cursor/Marker** button.
 - Click the **Insert** button.
2. Compress time in the *nWave* frame.

- You can add more compressed time ranges by repeating previous steps.
- You can also remove compressed time ranges by clicking a specific compressed time range in the main section in the *Compress Time Range* form and click the **Delete** button.
- When done, click the **Close** button.

Collapse Source Code

1. In the main window, enable all three options in the **Automatic Source Code Folding** field in **Tools -> Preferences -> Source Code** page -> **Code Folding** page.
2. In the source code frame, you see process and while blocks with a [-] and [+] next to the line number. Click on the minus [-] icon to collapse code.
3. If you want to expand all the collapsed code, invoke **View -> Source Code Folder -> Expand All in Design** in the main window.
4. To collapse all the code in a specific module, invoke **View -> Source Code Folder -> Collect All in Design** in the main window.

nAnalyzer - Extract Clock Tree

Note:

nAnalyzer is an optional Verdi module.

1. In the main window, invoke **Tools -> New Schematics -> Clock Tree**.
2. Click the **Import** button to open the *Import* form.
3. In the **Format** field change to **SDC**, and then select a SDC (Synopsys Design Constraint) file and then click **OK** to get clock sources from the SDC file.
4. Click the **OK** button to extract clock trees for the target clock sources.

nAnalyzer - Switching Reports

Note:

nAnalyzer is an optional Verdi module.

1. In the main window, invoke **Tools -> Switching Analysis -> New Query**.
 - a. In *Switching Analysis* form, working scope should be a scope that you want to focus on, or just use the top scope for everything, for example, `tb_CPUsystem.i_CPUsystem.i_CPU`;
 - b. Enable **Include Instances under the Hierarchy**.
 - c. Enable **Switching Activity Report** for the report type.
 - d. Click **OK**.
2. Click **OK** on the *Question* box to do *Behavior Analysis*.
3. In the *Switching Analysis Report* frame,
 - a. In the **Filters** tab, enter in a module name to search for, for example, enter: `*alu*` in the **Module** text field.
 - b. In the **Sorting Scheme** tab, select the **Sort By** option as **Name** and also enable the **Ascend** option.
 - c. Left-click the **Transition Count** column to sort the results by the number of transitions.
4. In the *Switching Analysis Report* frame, click the **New Query** button to open the *Switching Analysis* form.
 - a. In the *Switching Analysis* form, you can change the working scope to another level, for example, `tb_CPUsystem.i_CPUsystem.i_CPU.i_ALUB.i_alu`
 - b. Enable the **Peak Activity Report** option in the **Report Type** section.
 - c. Click **OK**.

Note:

The new *Switching Analysis Report* frame has different Time stamps with Transition Count. Also notice the “Time Period” at the top of the window should have some range, for example, “0 ~ 15000 x ns”.

- d. To find the location for the highest peak activity for an instance of time, double-click the row in the *Switching Analysis Report* frame that has the time that you want to see. This opens a new *Switching Analysis Report* frame but the **Time Period** is one snapshot in time, for example, “3000 ~ 3000 x ns”. This gives you all the locations in that time snapshot and their Transition Counts.

nAnalyzer - Timing Analysis

Note:

nAnalyzer is an optional Verdi module.

1. In the main window, invoke **File -> SDF -> Load SDF Files -> <your_sdf_file>.sdf**
2. In the main window, invoke **Source -> Find String**; enter the name of a register where you want to start. Select **Match Case** and **In All Files** options; double-click on desired result in the message frame at the bottom of the main window.
3. In the main window, click **New Schematic** icon.
4. In the *nSchema* frame, invoke **Trace -> Two Points**.
5. In the source code frame, drag and drop starting point output pin into **From** field of *Trace Two Points* form.
6. In the main window, invoke **Source -> Find String**; enter some name of a register where you want to end; Select **Match Case** and **In All Files** options; Double-click on desired result in the message frame at the bottom of the main window.
7. In the source code frame, drag and drop ending point output pin into **To** field of *Trace Two Points* form.
8. In *Trace Two Points* form, click the **Trace** button.
 - a. In *View Trace Result Schematic* frame, invoke **Schematic -> SDF Annotation**.
 - b. In *View Trace Result Schematic* frame, invoke **Schematic -> Delay Type** and **Schematic -> Delay Scale**.
 - c. In *View Trace Result Schematic* frame, invoke **Trace -> Shortest/Longest Path** and select **Longest**.
 - d. In the *nSchema* frame, right-click on cell to invoke **Show Cell Delay**.

nECO - Graphical Engineering Change Order Enhancement Tool - Modify Gate Level Netlist - Non-Freeze Silicon ECO

Note:

nECO is an optional Verdi module.

Note:

Frozen Silicon, spare cells need to be specified.

1. In the main window, invoke **File -> Import Path Data Files** to load the *PrimeTime* report file; an *Import Report File* form is opened.
2. In *Timing Report* form:
 - a. After **Enable Sorting**, select **Descending** or **Ascending**.
 - b. Select a path with the worst slack.
 - c. Click **File Viewer** in **Show On** section, and then click **Show** button.

-or-

 - d. Click on **nSchema** in **Show On** section, then click **Show** button at bottom.
3. In *nSchema*, enable **Schematic -> Auto Fit Found Object(s)**. Drag longest slack path on delay to *nSchema*
4. Fix the delay. In *nSchema*, find the cell output pin that may have the highest slack, and right-click to select **Connectivity**.
 - Assume the combinational logic on the load is too much, then, right-mouse-click and drag over all necessary and Shift click all Connected logic.
5. In *nSchema*, invoke **Tools -> New Schematic -> ECO Window for Selected Instance(s)**.
6. In the *nECO* frame, for example, to share a load of eight cells:
 - a. Shift-click on bottom four B inputs, then click **Disconnect Pin from Net** toolbar icon.
 - b. Find the cell that has too much load on it, right-click on cell to invoke **Copy Instance**, and then right-click to invoke **Paste Instance**.
 - c. Click on output of new cell to carry load and previous disconnected pins, and then click on **Make Connection** toolbar icon.
 - d. Connect all inputs of old loaded cell to inputs of new copy that helps share the load.
 - e. Click the **Keep Placement** toolbar icon to turn it off and on to reroute placement.
 - f. In *nECO*, invoke **File -> Commit Change**.
7. In the main window, change to the scope where the change was made by double-clicking, then search for the word Novas to see changes.
8. In the main window, invoke **File -> ECO -> Save ECO Netlist** can also save eco script.
 - To save *ECO Netlist* form, turn on **Affected Files Only**, then click **OK**.
9. In *nTrace*, invoke **File -> ECO -> ECO Report**.

- In *ECO Report* form, append eco0/eco.log to the **Full File Name** text field, then click **OK**.

```
% cd eco0; % emacs eco.log
```

Accelerate Simulation and Reduce Dumping Size with Siloti

1. Run a Siloti command as follows:

```
% vericom -f run.f &  
% esa -lib work -top top -db es -bas top -libPath work -bdb_file work  
&
```

- **-db**: To indicate the output should be an Essential Signal Database (ESDB), and specify the output ESDB file name.
 - **-bas <Design Top>**: Specify the design top for Behavior Analysis. If this option is not specified, *esa* finds a design top automatically.
 - **-libPath**: Specify the path to store BDB (Behavior Database). If this option is not specified, *./work* is the default path.
 - **-bdb_file**: Specify the BDB file name. If this option is not specified, *./work* is the default.
2. The design has been analyzed with Behavior Analysis to generate a minimal but sufficient set of signals to be dumped during simulation to provide 100% visibility during debug.
 3. Be sure to add the simulation runtime option **+fsdb+esdb** to specify the generated *esa* database when running your simulation:

```
% ./simv +fsdb+esdb="es"
```

4. Load design and essential signal FSDB file into the Verdi system:

```
% verdi -lib work -top top -esAuto -ssf esd.fsdb -bdb_load work.lib+  
+/work.bdb &
```

- **-esAuto**: Perform Data Expansion setup (auto time window mode) automatically after loading the design.
 - **-bdb_load**: Load the BDB file which has been generated by *esa* utility.
5. Enable Active Annotation by invoking **Source -> Active Annotation** in the main window to view data expansion results. Note that the value calculated by the Data Expansion engine is marked as purple.

6. Drag local signals to *nWave*. Drag an instance in the Verdi session.
7. Create a *Temporal Flow View* from this transition to view full capabilities of Verdi debug.

Resources

- **Verdi User Guide & Tutorial:** [\\$VERDI_HOME/doc/VerdiTut.pdf](#)
- **Siloti User Guide & Tutorial:** [\\$VERDI_HOME/doc/SilotiTut.pdf](#)
- **nECO User Guide & Tutorial:** [\\$VERDI_HOME/doc/nECO.pdf](#)
- **nAnalyzer User Guide & Tutorial:** [\\$VERDI_HOME/doc/nAnalyzer.pdf](#)
- **Verdi and Siloti Command Reference Manual:** [\\$VERDI_HOME/doc/verdi.pdf](#)
- **Demo cases:** [\\$VERDI_HOME/demo](#)